



TRUFLUX CONSULTING

Confidential White Paper

CODE MODERNISATION WITH AGENTIC AI

A Framework for Safely Transitioning Legacy Systems into the AI-Native Era

Prepared by Truflux Consulting

AI Strategy & Engineering Practice

April 2026

Author: Satya S Srinivasan is a seasoned technology leader with three decades of experience in consulting, digital transformation, cybersecurity, and AI-led innovation. Having worked with firms such as KPMG, Deloitte, Wipro, Accenture, and PwC, he brings deep expertise in helping organisations solve business problems through practical, scalable technology solutions.

Executive Summary

The emergence of agentic coding tools — AI systems that can autonomously plan, write, test, and deploy software — represents a pivotal shift in how organisations build and maintain technology. While these tools demonstrate remarkable capability in greenfield development, they continue to struggle with one of the most commercially critical challenges in enterprise software: understanding, modifying, and modernising existing production code.

This white paper, prepared by Truflux Consulting, introduces the **Truflux Code Modernisation Loop** — a structured, auditable framework that uses agentic AI not to patch old code, but to systematically re-derive it from first principles. By reverse-engineering production systems into living documentation (Business Requirements Documents, Technical Architecture, Non-Functional Requirements, and UI/UX Specifications), and then using those artefacts to drive AI-guided redevelopment, organisations can close the growing gap between legacy and agent-native codebases.

This paper examines the real-world lessons that make this gap urgent, the tools ecosystem that makes modernisation achievable today, and the testing rigour required before any re-derived system earns the right to production.

Table of Contents

1. The Agentic Coding Landscape — Promise and Peril
2. Production Failures Linked to Agentic Deployments
3. The Legacy Code Problem — Why Old Code Resists AI
4. The Truflux Code Modernisation Loop
5. Phase 1 — Documentation Extraction Tooling
6. Phase 2 — Agentic Coding Tooling
7. Phase 3 — Testing, Validation & Quality Assurance
8. Governance, Risk & Change Management
9. Recommendations for Clients
10. Conclusion

1. The Agentic Coding Landscape : Promise and Peril

Agentic coding has matured at extraordinary speed. In 2023, the category had barely emerged as a credible commercial product. By 2026, however, tools such as GitHub Copilot Workspace, Cursor, Devin, Claude Code, and Amazon Q Developer are being adopted across engineering organisations of every scale, from start-ups building their first products to Fortune 500 enterprises modernising large and complex software estates.

The appeal of this shift is easy to understand. Agentic coding tools do not merely suggest lines of code; they increasingly assist with interpreting requirements, generating scaffolding, writing functions, proposing tests, refactoring modules, debugging issues, and accelerating documentation. In practical terms, this means that a junior developer equipped with a capable agentic assistant can often deliver output that, only a short time ago, would have required the independent effort of a much more experienced engineer. Early adopter experience points to meaningful gains in productivity, with some teams reporting 30–60% reductions in time-to-first-commit for well-specified features and routine engineering tasks.

The benefits are particularly visible in greenfield environments. Where requirements are clear, architecture choices are straightforward, and the system is not burdened by years of accumulated complexity, agentic tools can perform impressively. They are well suited to building new services, creating APIs, scaffolding front-end components, generating integration layers, and rapidly producing working prototypes. In such conditions, they compress the gap between idea and implementation, allowing teams to move from concept to usable code at a pace that was difficult to imagine just a few years ago.

This has created a powerful narrative in the market. Leaders see the possibility of dramatically higher engineering throughput. Product teams see faster experimentation. Start-ups see a way to do more with smaller teams. Enterprises see a path to improved developer productivity, shorter release cycles, and lower delivery friction. In the best cases, agentic coding is not just a productivity enhancer; it becomes a force multiplier that changes how software teams organise work, allocate senior talent, and think about delivery capacity.

Yet the strongest results tend to appear where the work is well bounded and the context is relatively clean. The more explicit the requirements, the better the architecture hygiene, and the lower the dependency complexity, the better these tools tend to perform. This is why they shine in building new systems, isolated services, and clearly defined feature increments. In these scenarios, agentic coding can feel transformative.

The real test, however, begins when these tools encounter the reality of most enterprises: sprawling legacy estates, undocumented business rules, tightly coupled systems, brittle integrations, inconsistent coding standards, and years of technical debt. That is where the promise of agentic coding remains substantial, but where its effectiveness becomes far more dependent on context, governance, and the underlying health of the codebase itself.

Agentic Tools: Strengths vs. Weaknesses

Characteristic	Strengths	Weaknesses
 Development	Greenfield development from structured specifications	Understanding deeply contextualised business logic
 Boilerplate	Boilerplate generation: CRUD APIs, data models	Navigating implicit architectural decisions
 Code	Code explanation and documentation	Interpreting undocumented tribal knowledge
 Testing	Generating unit and integration tests	Safely deploying changes to production
 Refactoring	Refactoring isolated, well-typed functions	Managing stateful, distributed systems

1.1 Where Agentic Tools Excel Today

- Greenfield development from structured specifications (BRDs, user stories, API contracts)
- Boilerplate generation: CRUD APIs, data models, test scaffolding, CI/CD pipelines
- Code explanation and documentation of known, self-contained modules
- Generating unit and integration tests for newly written, clean code
- Refactoring isolated, well-typed functions with clear input/output contracts

1.2 Where Agentic Tools Consistently Struggle

- Understanding deeply contextualised business logic embedded in decade-old code
- Navigating implicit architectural decisions made due to infrastructure constraints that no longer exist
- Managing stateful, distributed systems with complex failure modes
- Interpreting undocumented tribal knowledge — the "why" behind non-obvious implementations
- Safely deploying changes to production systems with high blast radius

Point of View:

When advising clients, Truflux positions agentic tools not as a replacement for engineering judgment, **but as a force multiplier** that works best when given well-structured inputs. The Modernisation Loop is specifically designed to create those inputs from existing systems.

†Representative agentic coding products entered preview or broader market adoption across 2024–2025, including GitHub Copilot Workspace, Amazon Q Developer, Claude Code, Devin, and Cursor.

†Early studies and vendor-reported enterprise results suggest meaningful productivity gains from AI-assisted development, including faster task completion, 25% faster initial development in some customer reports, and case examples showing 40–50% acceleration in modernization workstreams.

†Published industry analyses suggest that gen-AI-enabled engineering performs best when work is clearly specified, well-bounded, and governed with strong human oversight and contextual documentation.

2. Production Failures Linked to Agentic Deployments

The most important case for the Truflux Code Modernisation Loop is not theoretical — it is demonstrated by a growing body of production incidents where AI-assisted or AI-driven deployments caused significant commercial harm.

2.1 Amazon AWS — AI-Assisted Deployment Outage (2024)

In late 2024, Amazon experienced a notable service disruption attributed in part to an AI-assisted configuration change that was applied without adequate human review. The incident involved an agentic tool generating infrastructure-as-code changes that were syntactically valid but semantically incorrect for the specific network topology in use. The change was staged, reviewed at surface level, and deployed. The resulting cascading failure affected multiple AWS regions for several hours, impacting downstream customers at scale.

The core failure was not the AI tool's capability — the generated code was correct for the most common configuration pattern. The failure was contextual blindness: the tool had no mechanism to understand that this particular environment had non-standard routing rules established years earlier to accommodate a regulatory constraint.

Point of View:

Agentic tools optimize for the most probable interpretation of a specification. Legacy systems often encode the improbable — edge cases, regulatory accommodations, and historical workarounds that are invisible to a model trained on general code patterns.

2.2 UK NHS Digital – AI Code Merge Incident (2024–2025)

A regional NHS Digital implementation team deployed an AI-assisted code merge that incorrectly consolidated two patient data pathway modules. The AI tool, tasked with reducing duplication, identified structurally similar code in both modules and merged them. What the tool did not recognise was that the apparent duplication was intentional: one pathway handled adult patient records, and the other handled child records, with subtly different data retention and consent rules mandated by the Children Act.

The result was a period during which consent checks for a subset of child records defaulted to adult-pathway logic. The incident required a full data audit, regulatory notification, and emergency rollback — at significant cost and reputational risk.

2.3 Financial Services – Automated Trading System Regression (2025)

A major European bank piloting an agentic coding tool for its trading platform modernisation programme experienced a severe regression when the tool refactored a risk calculation module. The original module contained a deliberate, undocumented computational approximation chosen for latency reasons under specific market data volume conditions. The agentic tool replaced it with a mathematically "correct" calculation that was substantially slower, causing the system to breach latency SLAs during peak market hours. Automated circuit breakers triggered, and the platform was offline for 47 minutes during active trading.

2.4 E-Commerce Platform – AI-Driven A/B Test Deployment Failure (2025)

A global e-commerce operator using an agentic deployment pipeline deployed a set of AI-generated checkout flow changes. The changes were tested against synthetic user behaviour data generated by the same AI system — creating a closed evaluation loop

that confirmed AI assumptions rather than testing against real-world edge cases. The deployed flow broke payment processing for users with non-standard shipping addresses in three Asia-Pacific markets. The incident cost an estimated \$4.2M in lost transactions over 11 hours before detection and rollback.

2.5 Common Root Causes Across Incidents

Root Cause	Frequency	Mitigation in Modernisation Loop
Contextual blindness (implicit business rules not visible in code)	Very High	Phase 1 documentation extraction makes implicit rules explicit
Self-referential testing (AI tests AI's own assumptions)	High	Phase 3 requires production comparison testing as independent oracle
No requirements traceability	High	RTM enforces every feature links to a documented requirement
Inadequate human review gates	Medium	Loop mandates human sign-off at each phase boundary
Legacy context loss	High	BRD + Technical Spec capture the "why" before redevelopment begins

3. The Legacy Code Problem : Why Old Code Resists AI

Legacy systems are not simply old software. They are archaeological records of business decisions, technical constraints, team capabilities, and market conditions that no longer exist. Agentic tools trained on open-source code repositories have excellent syntactic and structural intuition – but almost no ability to reconstruct the commercial and regulatory context that shaped a given system.

3.1 The Three Layers of Legacy Context

Layer 1: Technical Debt as Intentional Architecture

Much of what appears to be technical debt is in fact deliberate trade-offs. A synchronous API call where asynchronous would seem obvious may exist because a downstream system's async callback handling was unreliable in 2015 and the team never revisited it after the downstream system was replaced. An agentic tool sees the synchronous call and replaces it – breaking the assumption that no longer needs to hold, but without verifying that the downstream system now handles async reliably.

Layer 2: Embedded Regulatory and Compliance Logic

Financial services, healthcare, and government-adjacent systems routinely embed compliance logic in ways that are structurally indistinguishable from general business logic. GDPR data-handling paths, FCA reporting windows, HIPAA de-identification routines — these are often woven into application code without clear demarcation. Agentic tools have no regulatory awareness; they optimize for clean code, not compliant code.

Layer 3: Operational Knowledge

Production behaviour teaches things that code alone cannot. A module that runs a nightly batch job at 2:47 AM (not 2:00 AM as might seem logical) does so because 2:47 AM was empirically discovered to be the quietest window on the database cluster. This knowledge lives in the minds of on-call engineers, not in comments or documentation.

3.2 The Widening Gap

As agentic tools improve, newly developed systems will be written in increasingly structured, self-documenting ways that make them more amenable to future AI modification. Legacy systems will not benefit from this — the gap between AI-native and human-native codebases will widen over time, not narrow.

Organisations that do not actively modernise legacy systems risk a two-tier architecture: a growing portfolio of AI-native services that can be iterated rapidly, alongside an ageing core of legacy systems that can only be touched by the dwindling number of engineers who understand them.

Strategic Risk

By 2028, Truflux estimates that organisations without an active code modernisation programme will face a 3-5x differential in iteration speed between their legacy and modern systems — creating a compounding competitive disadvantage.

4. The Truflux Code Modernisation Loop

The Truflux Code Modernisation Loop is a structured, repeatable process for safely re-deriving production systems using agentic AI. It is not a one-time project — it is a continuous programme that cycles through a portfolio of legacy systems, each time producing a cleaner, more maintainable, AI-native successor.

4.1 The Five Phases

Phase	Activity	Output
Phase 0: Inventory & Prioritisation	Catalogue legacy systems; score by complexity, business criticality, and modernisation risk	Modernisation Roadmap
Phase 1: Documentation Extraction	Derive BRD, Tech Spec, NFR, UI/UX Spec from deployed production system	Living Documentation Suite
Phase 2: Prompt Engineering	Convert documentation into structured agentic coding prompts and acceptance criteria	Prompt Library + Acceptance Criteria
Phase 3: Agentic Redevelopment	Use agentic tools to re-derive the codebase against the documentation	Candidate Codebase
Phase 4: Validation & Testing	Cyclomatic complexity analysis, RTM validation, production comparison testing	Validated Release Candidate
Phase 5: Deployment & Monitoring	Blue/green or canary deployment with real-time production comparison	Modernised Production System

4.2 The Loop Principle

The "Loop" in the Modernisation Loop refers to the continuous cycle: after a system is modernised, the modernised system itself becomes the baseline for future modernisation cycles. Each cycle produces increasingly cleaner, better-documented, more AI-native code – compounding returns over time.

This is the critical differentiator from one-time rewrite projects, which historically fail at high rates because they attempt to capture all requirements upfront and execute in a single long arc. The Loop is incremental, iterative, and designed to deliver value at every phase boundary.

5. Phase 1 – Documentation Extraction Tooling

The most critical phase of the Modernisation Loop is Phase 1. The quality of the documentation produced here determines the quality of everything that follows. This is where the implicit knowledge embedded in legacy code is made explicit, structured, and auditable.

5.1 Business Requirements Document (BRD) Tools

5.1.1 AI-Assisted Code Analysis for BRD Generation

Modern AI-powered tools can ingest entire codebases and produce initial BRD drafts by analysing function names, API contracts, database schemas, user-facing strings, and business logic patterns.

Tool	Description & BRD Use Case
Swimm	Code-coupled documentation platform. Analyses living codebases and generates contextual documentation that stays in sync with code changes. Excellent for generating initial feature inventories and workflow descriptions.
CodeSee	Produces visual maps of codebase architecture, surfacing user journeys and data flows that inform BRD user story development.
Bito AI	AI code review and explanation tool. Can be prompted to produce natural-language descriptions of business intent from code modules – primary input for BRD authoring.
Apiiro	Risk and change management platform that analyses code and infrastructure to surface business-critical components, useful for scoping BRD coverage.
Claude (Anthropic)	Large codebase analysis via extended context. Can process tens of thousands of lines and produce structured BRD sections with business logic summaries, exception handling documentation, and user-journey extraction.

5.2 Technical Architecture & Specification Tools

5.2.1 Automated Architecture Recovery

Technical specifications must capture not just what the system does, but how it is structured and why architectural decisions were made.

Tool	Description & Tech Spec Use Case
Structurizr	C4 model-based architecture diagramming tool. Can be used to capture recovered architecture and produce standardised technical documentation (Context, Container, Component, Code diagrams).
Backstage (Spotify)	Developer portal and service catalogue. Used to document discovered service dependencies, API contracts, and

Tool	Description & Tech Spec Use Case
	ownership — foundational for Technical Architecture documents.
Archium / Jarviz	Automated dependency and architecture analysis tools. Produces component relationship maps and identifies architectural patterns (layered, event-driven, microservices) from code analysis.
SonarQube	Code quality and architecture analysis. Identifies structural violations, complexity hotspots, and coupling that must be documented before redevelopment.
AWS Application Composer / Azure Application Insights	Cloud-native tools for recovering infrastructure architecture from deployed environments — critical when infrastructure state has drifted from any existing documentation.

5.3 Non-Functional Requirements (NFR) Extraction

NFRs — performance, availability, security, scalability, compliance — are almost never explicitly documented in legacy systems. They must be empirically measured from production behaviour.

Tool	Description & NFR Use Case
Datadog / New Relic / Dynatrace	Production observability platforms. Used to establish empirical performance baselines: P50/P95/P99 latency, throughput, error rates, and resource consumption patterns that form quantitative NFRs.
OWASP Dependency-Check	Security vulnerability analysis of existing dependencies — used to document security NFRs and inform the security posture requirements for the re-derived system.
k6 / Gatling	Load testing tools used to characterise existing system performance under stress — essential for defining scalability NFRs that the modernised system must meet or exceed.
AWS Trusted Advisor / Azure Advisor	Cloud cost and performance advisors that surface operational NFRs (cost efficiency, redundancy, region coverage) from deployed infrastructure.
Prometheus + Grafana	Open-source observability stack for capturing long-running production metrics to establish SLA baselines for NFR documentation.

5.4 UI/UX Specification Extraction

5.4.1 Capturing Existing User Experience

For client-facing systems, the UI/UX specification must capture not just the visual design but the interaction patterns, accessibility standards, and user journey logic that users depend on.

Tool	Description & UI/UX Use Case
Figma / FigJam	Reverse-engineer existing UIs into Figma by screenshotting and rebuilding component libraries. Enables design system documentation before redevelopment begins.
Maze / FullStory / Hotjar	Behavioural analytics tools that document how users actually interact with the existing system — surfacing interaction patterns that may not be obvious from code alone.
Playwright (trace viewer)	End-to-end testing tool with visual trace capture. Used to document existing user flows as visual specifications that form the UI/UX acceptance criteria.
Zeplin / Storybook	Design-to-developer handoff tools used to document and componentise the existing UI design system for specification purposes.
axe DevTools	Accessibility analysis tool. Documents the current accessibility compliance level (WCAG 2.1/2.2) — a key NFR that the modernised system must maintain or improve.

5.5 Converting Documentation to Prompts

Once the documentation suite is complete, it must be transformed into structured prompts and acceptance criteria that agentic coding tools can act upon. This is a discipline in itself — prompt engineering for code generation is materially different from prompt engineering for text generation.

Tool / Technique	Description
Anthropic Claude (Projects)	Structured prompt development using Claude's Projects feature to maintain context across the documentation-to-prompt conversion process. System prompts encode the architecture and NFRs; user prompts encode individual feature specifications.

Tool/ Technique	Description
PromptLayer / LangSmith	Prompt management and versioning platforms. Used to maintain, test, and iterate prompt libraries that encode BRD requirements as structured coding instructions.
OpenAPI / AsyncAPI Specification	API specification formats used to encode Technical Architecture requirements as machine-readable contracts that directly feed agentic coding tool context.
Gherkin / BDD Frameworks	Behaviour-Driven Development syntax (Given/When/Then) converts BRD user stories into structured acceptance criteria that serve simultaneously as human specifications and automated test definitions.
Custom Prompt Templates	Truflux-developed prompt templates that encode BRD sections, NFRs, and architectural constraints into structured formats optimised for each target agentic coding tool.

6. Phase 2 – Agentic Coding Tooling

With a complete documentation suite and structured prompt library, the organisation is ready to engage agentic coding tools for the re-derivation phase. Tool selection at this stage is not merely a preference decision – it has material impact on output quality, auditability, and integration with the testing pipeline.

6.1 Agentic Coding Tool Landscape (2026)

Tool	Strengths	Best For in Modernisation Loop
Claude Code (Anthropic)	Extended context, deep reasoning, excellent architectural understanding, strong at navigating complex specifications	Architecture-faithful redevelopment; processing large documentation suites; reasoning about trade-offs
GitHub Copilot Workspace	Deep GitHub integration, issue-to-code pipeline, strong for iterative PR-based workflows	Teams using GitHub for version control; incremental module-by-module redevelopment
Cursor	IDE-native experience, codebase-aware, excellent for human-in-the-loop development with AI pair programming	Senior engineers guiding AI redevelopment of complex modules with domain expertise

Tool	Strengths	Best For in Modernisation Loop
Devin (Cognition)	Fully autonomous agent with long-horizon task execution, web browsing, and self-debugging capability	Well-specified modules with clear acceptance criteria; tasks requiring research during development
Amazon Developer Q	AWS-native, strong infrastructure-as-code generation, integrated security scanning	AWS-hosted legacy systems; infrastructure modernisation alongside application redevelopment
Replit Agent	Full-stack deployment-aware development, strong for rapid prototyping and validation	Early validation sprints to test documentation quality before full redevelopment commitment

6.2 Agentic Coding Governance Principles

Agentic coding tools in the Modernisation Loop must operate under strict governance constraints. Based on Truflux's experience and the incident analysis in Section 2, the following principles are non-negotiable:

1. Every agent task must be scoped to a single module or feature boundary. No agent task should span architectural layers.
2. Agents must be given explicit negative constraints alongside positive specifications — what the system must NOT do is as important as what it must do.
3. All agent-generated code must be committed to version control before any testing begins.
4. Avoiding in-memory evaluation of agent output reduces non-deterministic behavior, improves reliability, and minimizes security risks in production-grade agentic systems.
5. Human review is mandatory at module boundaries. Agents do not have the authority to chain module completions autonomously.
6. Agent context must be reset between modules. Accumulated context from previous module development can pollute architectural decisions in subsequent modules.

7. Phase 3 – Testing, Validation & Quality Assurance

The testing phase of the Modernisation Loop is where the modernised codebase earns – or fails to earn – the right to production. This phase is deliberately more extensive than conventional software testing, because the re-derived system is being held to a higher standard: it must not only be functionally correct, it must be provably equivalent to the production system it is replacing.

7.1 Cyclomatic Complexity Analysis

What It Is

Cyclomatic complexity (CC) measures the number of linearly independent paths through a program's source code. Developed by Thomas McCabe in 1976, it remains one of the most reliable predictors of defect density, test coverage difficulty, and maintainability.

Target Thresholds

CC Score	Classification	Action Required
1-10	Low complexity – simple, linear	Standard unit testing; no remediation needed
11-20	Moderate complexity	Ensure branch coverage; document complex paths
21-50	High complexity	Mandatory refactoring before production deployment
50+	Untestable – extreme complexity	Module-level redesign required; escalate to architecture review

Why It Matters in Modernisation

The Modernisation Loop has a specific CC mandate: the re-derived system must have measurably lower average cyclomatic complexity than the legacy system. If the agentic tool has reproduced the structural complexity of the original code (a common failure mode where agents faithfully replicate code structure rather than rethinking it), the modernisation has not achieved its objective.

Tool	Use in CC Analysis
SonarQube	Primary CC measurement platform. Enforces CC thresholds as quality gate blockers in CI/CD pipelines; historical comparison against legacy baseline.
Lizard	Lightweight, language-agnostic CC analyser. Used for rapid pre-commit CC checks before full SonarQube analysis.
CodeClimate	CC analysis with maintainability scoring and trend tracking – useful for demonstrating improvement trajectory to stakeholders.
NDepend (.NET)	Deep static analysis for .NET systems including CC, coupling metrics, and architectural validation rules.
PMD / Checkstyle (Java)	Java-specific static analysis tools with CC measurement and configurable rule sets for enforcing complexity thresholds.

7.2 Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix is the governance backbone of the Modernisation Loop's testing phase. It ensures that every requirement documented in the BRD and Technical Specification is explicitly covered by at least one test case, and that no test case exists without a traceable requirement.

RTM Structure

Column	Contents	Owner
Requirement ID	Unique identifier from BRD or Tech Spec	Business Analyst / Consultant
Requirement Description	Plain-language statement of the requirement	Business Analyst
Priority	Must Have / Should Have / Could Have (MoSCoW)	Product Owner / Consultant
Test Case ID(s)	Linked automated test case identifiers	QA Lead
Test Type	Unit / Integration / E2E / Performance / Security	QA Lead
Test Status	Pass / Fail / Blocked / Not Run	QA Engineer
Production Comparison	Does modernised output match production system output?	QA Engineer

Column	Contents	Owner
Sign-off	Human approval of requirement closure	Consultant / Business Owner

RTM Tooling

Tool	RTM Use Case
Jira + Xray	Industry-standard RTM integration. Xray links Jira requirements to automated test results, producing live traceability reports. Recommended for enterprise clients.
TestRail	Dedicated test management platform with native RTM support, customisable traceability reports, and milestone tracking aligned to Modernisation Loop phases.
Zephyr Scale	Jira-native test management with RTM dashboards, coverage reports, and integration with popular CI tools.
qTest (Tricentis)	Enterprise-grade test management with hierarchical RTM, supporting complex traceability across BRD, Tech Spec, and NFR requirements simultaneously.
Custom RTM (Confluence + Automation)	For teams with existing Confluence-based documentation, automated RTM population from Jira using Confluence macros and Jira automation rules.

7.3 Feature Validation Testing

Feature validation confirms that every business feature identified in the BRD is present, correctly implemented, and behaves identically to the production system. This is distinct from unit testing (which validates code correctness) and integration testing (which validates system connectivity) – feature validation validates business intent.

Feature Validation Toolkit

Tool	Use Case
Playwright	End-to-end browser automation for UI feature validation. Captures and replays user journeys documented in the UI/UX Specification. Produces visual diffs against production screenshots.
Cypress	Component and E2E testing with real-time visual feedback. Particularly strong for React and modern frontend stacks.

Tool	Use Case
Cucumber (with Gherkin)	BDD framework that executes Gherkin acceptance criteria directly as automated tests – closing the loop between BRD requirements and test execution.
Postman / Pactflow	API feature validation and contract testing. Ensures API behaviour of the modernised system is contract-compatible with existing integrations.
Storybook + Chromatic	Visual regression testing for UI components – confirms that re-derived frontend components are visually identical to production counterparts.

7.4 Production Comparison Testing

Production comparison testing is the most distinctive element of the Modernisation Loop's QA framework. Unlike shadow testing (which runs new code alongside production and monitors for errors), production comparison testing systematically validates that the modernised system produces identical outputs to the production system for identical inputs – across a statistically significant sample of real production traffic.

Implementation Pattern

1. Traffic mirroring: Route a configurable percentage of production traffic simultaneously to both the legacy system and the modernised system.
2. Response comparison: Capture and compare API responses, database mutations, event emissions, and side effects. Flag any divergence – including differences in response timing that could indicate an NFR violation.
3. Edge case harvesting: Use production traffic logs to identify and replay edge cases – the long tail of unusual inputs that synthetic tests almost never cover.
4. Divergence analysis: Every divergence is classified as: (a) defect in modernised system, (b) intentional improvement, or (c) undocumented legacy behaviour that must be documented and validated.
5. Confidence threshold: Production deployment proceeds only when divergence rate falls below a client-agreed threshold (typically <0.01% of transactions).

Tool	Use Case in Production Comparison
Diffy (Twitter/X open source)	Purpose-built production comparison tool. Routes requests to two backends and automatically diffs responses, with noise filtering for timestamps and session tokens.
Envoy Proxy + Shadow Traffic	Infrastructure-level traffic mirroring with response capture. Language-agnostic production comparison at the network layer.
AWS CodeDeploy (linear deployment)	Gradual traffic shifting with automatic rollback on alarm – supports incremental production comparison at controlled exposure levels.
LaunchDarkly	Feature flag platform enabling traffic-based comparison testing with precise population control and instant rollback capability.
Gatling (replay)	Production traffic replay tool – captures production request logs and replays them against the modernised system for comparison without live traffic exposure.

7.5 Security & Compliance Testing

Re-derived systems must meet or exceed the security posture of the systems they replace. The following security testing is mandatory in the Modernisation Loop:

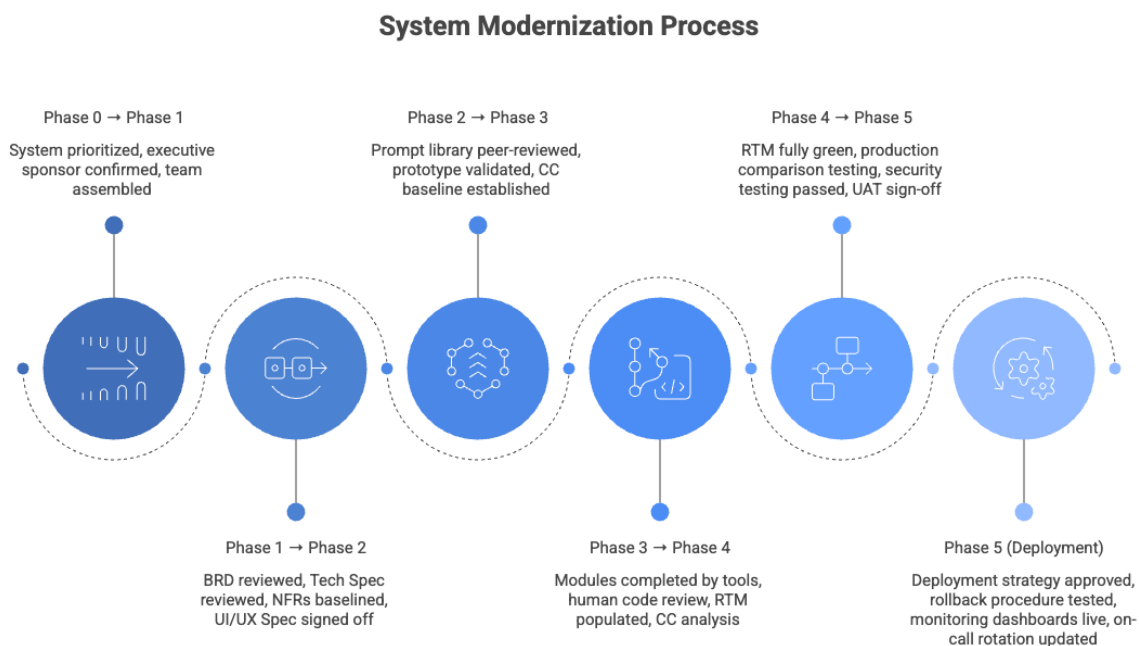
- SAST (Static Application Security Testing): Checkmarx, Semgrep, or Snyk Code – run on every PR to the modernised codebase
- DAST (Dynamic Application Security Testing): OWASP ZAP or Burp Suite – full scan before any production comparison testing begins
- Dependency scanning: Snyk or OWASP Dependency-Check – all third-party dependencies audited against known vulnerability databases
- Compliance validation: Automated compliance checks (SOC 2, GDPR, HIPAA as applicable) using tools like Vanta, Drata, or Secureframe
- Penetration testing: Mandatory third-party pen test before production deployment for Tier 1 systems

8. Governance, Risk & Change Management

8.1 The Human-in-the-Loop Mandate

The Truflux Code Modernisation Loop is explicitly not a fully automated pipeline. At every phase boundary, human sign-off is required. This is not a conservative hedge — it is an architectural decision informed by the failure patterns documented in Section 2. Autonomous, closed-loop AI systems in production are the primary cause of agentic deployment failures. The Loop is designed to prevent this by creating mandatory human checkpoints.

8.2 Phase Gate Criteria



Gate	Minimum Criteria for Progression
Phase 0 → Phase 1	System prioritised on Modernisation Roadmap; executive sponsor confirmed; modernisation team assembled
Phase 1 → Phase 2	BRD reviewed and approved by business owner; Tech Spec reviewed by senior engineer; NFRs baselined from production measurements; UI/UX Spec signed off by design lead
Phase 2 → Phase 3	Prompt library peer-reviewed; initial prototype validated against at least 3 core BRD requirements; CC baseline established for legacy system
Phase 3 → Phase 4	All modules completed by agentic tools; human code review complete; RTM populated with ≥95% requirement coverage; CC analysis shows improvement vs. legacy baseline

Gate	Minimum Criteria for Progression
Phase 4 → Phase 5	RTM fully green; production comparison testing complete with divergence below agreed threshold; security testing passed; business owner UAT sign-off received
Phase 5 (Deployment)	Blue/green or canary deployment strategy approved; rollback procedure tested; monitoring dashboards live; on-call rotation updated

8.3 Risk Register

Every Modernisation Loop engagement must maintain a live risk register. Core risks identified across Truflux engagements include:

- Documentation incompleteness: Failure to capture all business logic in Phase 1 remains the highest-probability risk. Mitigated by domain expert interviews alongside automated extraction.
- Scope creep: The modernisation process often surfaces improvement opportunities. These must be logged and deferred to a post-modernisation roadmap — the Loop's objective is equivalence, not enhancement.
- Production divergence underestimation: Edge cases in production traffic may not emerge during the comparison testing window. Mitigated by extended parallel operation periods for Tier 1 systems.
- Stakeholder resistance: Engineering teams may resist having their systems "replaced by AI." Change management and transparent communication of human governance are essential.

9. Recommendations for Clients

Based on Truflux's experience deploying and advising on agentic coding programmes, the following recommendations are offered for clients embarking on a code modernisation journey:

9.1 Start with a Pilot System

Do not begin with your most critical system. Select a system that is: (a) meaningful enough that the modernisation delivers real value, (b) small enough that Phase 1 documentation can be completed in 4–6 weeks, and (c) low enough business risk that a deployment failure has manageable consequences. Use the pilot to calibrate your team's capability with the Loop before scaling.

9.2 Invest Disproportionately in Phase 1

The temptation is to race to the "exciting" parts — the agentic coding and the deployment. Resist this. Phase 1 documentation extraction is the most important investment you will make. Every hour spent making implicit knowledge explicit in Phase 1 saves multiple hours of debugging in Phase 3 and Phase 4. Truflux recommends a minimum 40% of total programme effort be allocated to Phase 1.

9.3 Build a Bespoke Prompt Library

Generic prompts produce generic code. Your agentic coding tools must be given context-rich, constraint-explicit prompts that encode your specific architecture, your NFRs, your team's coding standards, and your regulatory environment. Investing in a well-engineered prompt library is a durable asset — it improves with each Loop cycle.

9.4 Never Skip Production Comparison Testing

Production comparison testing is the non-negotiable quality gate of the Modernisation Loop. It is also the step most frequently proposed for elimination when schedules are tight. Do not allow this trade-off. Every incident described in Section 2 occurred in programmes that short-cut their validation testing. Production comparison testing is not a cost — it is the insurance policy that makes the entire programme safe.

9.5 Treat Modernisation as a Programme, Not a Project

A project has a start date and an end date. A programme is continuous. The Modernisation Loop is designed to run perpetually — cycling through your application portfolio, with each cycle producing progressively cleaner systems. Organisations that treat this as a one-time project will find themselves back in the same position in five years. Establish the Loop as a standing engineering programme with a dedicated team and board-level visibility.

9.6 Measure and Report Improvement

Modernisation programmes are often invisible to business stakeholders — the modernised system looks and behaves identically to the one it replaced. Make the value visible by tracking and reporting: cyclomatic complexity reduction, defect density trends, deployment frequency, time-to-recover from incidents, and engineering time allocated to maintaining legacy systems. These metrics build the business case for continued investment.

10. Conclusion

The gap between legacy codebases and AI-native systems is not a temporary condition that will resolve itself as agentic tools improve. It is a structural divergence that will widen unless organisations take deliberate action to close it.

The Truflux Code Modernisation Loop provides that action: a structured, safe, continuously improving process that uses the power of agentic AI under rigorous human governance to systematically re-derive production systems — making them AI-native, lower in complexity, and equipped for the development velocity that the next decade will require.

The incidents described in Section 2 are not cautionary tales about the dangers of AI in software development. They are cautionary tales about the dangers of AI without structure, without documentation, without traceability, and without the production comparison testing that allows teams to know — not hope — that a modernised system is safe to deploy.

With the right framework, agentic AI does not introduce risk into software modernisation. It reduces it.

Next Steps

Truflux Consulting offers a Code Modernisation Readiness Assessment — a structured two-day engagement that produces a prioritised Modernisation Roadmap and a pilot system recommendation for your organisation. Contact your Truflux account manager or reach out via contact@trufluxtech.com to schedule.

Appendix A – Tool Reference Summary

The following table provides a consolidated reference of all tools referenced in this white paper, organised by Modernisation Loop phase.

Phase	Tool / Platform	Primary Purpose
Phase 1 – BRD	Swimm	Code-coupled documentation generation
Phase 1 – BRD	CodeSee	Codebase visual mapping for user journey extraction
Phase 1 – BRD	Bito AI	Natural-language business intent extraction from code
Phase 1 – BRD	Claude (Anthropic)	Large codebase analysis and BRD section generation
Phase 1 – Tech Spec	Structurizr	C4 architecture documentation
Phase 1 – Tech Spec	Backstage	Service catalogue and dependency documentation
Phase 1 – Tech Spec	SonarQube	Structural complexity and architecture analysis
Phase 1 – NFR	Datadog / Dynatrace	Production performance baseline measurement
Phase 1 – NFR	k6 / Gatling	Load testing for scalability NFR establishment
Phase 1 – UI/UX	Figma	UI reverse-engineering and design specification
Phase 1 – UI/UX	FullStory / Hotjar	User behaviour analytics for interaction specification
Phase 1 – UI/UX	Playwright	User journey capture and documentation
Phase 1 – Prompts	PromptLayer / LangSmith	Prompt versioning and management
Phase 1 – Prompts	Cucumber / Gherkin	BDD acceptance criteria as structured prompts
Phase 2 – Coding	Claude Code	Architecture-faithful re-derivation
Phase 2 – Coding	GitHub Copilot Workspace	PR-based iterative redevelopment

Phase	Tool / Platform	Primary Purpose
Phase 2 – Coding	Cursor	Human-in-the-loop AI pair programming
Phase 2 – Coding	Devin	Autonomous long-horizon task execution
Phase 2 – Coding	Amazon Q Developer	AWS-native redevelopment
Phase 3 – CC	SonarQube	Cyclomatic complexity measurement and quality gates
Phase 3 – CC	Lizard	Lightweight pre-commit CC analysis
Phase 3 – RTM	Jira + Xray	Requirements traceability management
Phase 3 – RTM	TestRail	Test case management and RTM reporting
Phase 3 – Feature	Playwright / Cypress	End-to-end feature validation
Phase 3 – Feature	Postman / Pactflow	API contract and feature validation
Phase 3 – Prod Comp	Diffy	Response comparison between legacy and modernised systems
Phase 3 – Prod Comp	Envoy + Shadow Traffic	Network-layer traffic mirroring
Phase 3 – Prod Comp	LaunchDarkly	Feature flag-controlled traffic comparison
Phase 3 – Security	Snyk / Semgrep	SAST and dependency vulnerability scanning
Phase 3 – Security	OWASP ZAP	Dynamic application security testing
Phase 5 – Deploy	AWS CodeDeploy / LaunchDarkly	Gradual deployment with automated rollback

This document is confidential and prepared solely for educational purposes. Reproduction or distribution without the prior written consent of Truflux Consulting is strictly prohibited.

© 2026 Truflux Consulting. All rights reserved.

This document reflects the author's understanding and is intended for educational reference only. Any projects referenced herein are illustrative in nature and do not imply that they were executed, managed, or personally experienced by the author.